Exercise: Define a Class

Learning Objectives

You have encountered the basic principles of Object Oriented programming and in some preliminary ways demonstrated how the different principles can be put into practice with the help of classes, the building blocks of OOP. Let us now look at the structure of these classes.

Here you will learn how to create classes and objects with the help of examples. Let's first look at the basic members of a class. These can be the attributes or the data members, the methods, and additionally the comments that you can include. These members can be shown with the help of an example below. Let us imagine you want to make a class of some house. You begin by creating a class for it.

Example 1

```
class House:
    '''
    This is a stub for a class representing a house that can be used to create objects and evaluate different metrics that we may require in constructing it.
    '''
    num_rooms = 5
    bathrooms = 2
    def cost_evaluation(self):
        print(self.num_rooms)
        pass
        # Functionality to calculate the costs from the area of the house
```

In the code above, you start with a multiline comment, which alternatively can also be called a docstring (''' enclosed comments '''). In the next line you have the class definition, followed by a couple of data members or attributes: **`num_rooms`** and **`bathrooms`**. This is then followed by a function definition, which is empty except for the **`pass`** keyword that basically signals Python to continue execution without throwing an error. The last line in the code block is the single-line comment preceded by **#**.

The code completely defines the class and functions present inside it, but it is effectively not useful unless you call or instantiate it. You can do this by one of the two ways: Calling the class directly
Instantiating an object of that class

You can add a few lines of code below your code that will call the variable num_rooms on the house object and the House class after we create a house object from House class:

```
house = House()
print(house.num_rooms)
print(House.num_rooms)
```

The effective output for this will be:

```
5
5
```

To follow up with this example, add few more lines to this code and see the output, this time after you have updated the num_rooms variable called on house object to 7:

```
house.num_rooms = 7
print(house.num_rooms)
print(House.num_rooms)
```

The new output this time will be:

```
5
5
7
5
```

What has happened in the code above is, you have created an instance of a class called **`house`** and then modified the attribute for that instance with a value of 7. It updates the value of the instance attribute, but not the class attribute. So the num_rooms attribute of the class remains unchanged as 5, but the instance attribute associated with house object changes to 7. Let's now insert an alternate piece of code in this.

This time, instead of an instance attribute, you will modify the class attribute by directly calling it over the class as follows:

```
House.num_rooms = 7
print(house.num_rooms)
print(House.num_rooms)
```

The output for it will be:

```
5
5
7
7
```

You will notice that the changes on a class attribute will affect even the instances that you will create over it. Also note the use of the keywork *self* in this example. **`self`** is a convention in Python, and you may use any other word in its place, but as a practice, it is easy to recognize. **`self`** here is passed inside the method **`cost_evaluation()`** as it is an instance method and facilitates the method to point to any instance of the House when that method is called. It should be noted how any number of parameters can be passed to these instance methods but the first one is always the reference to the instance of that class.

You can interact and run the entire program that you just saw in the code block below:

Completed **Go to next item**

Like **Dislike** **Report an issue**